```
  end

  private
    # Use callbacks to share common setup or constraints between actions.
    def set_product
      @product = Product.find(params.expect(:id))
    end

    # Only allow a list of trusted parameters through.
    def product_params
      params.expect(product: [ :title, :description, :image, :price ])
    end
end
```

Notice how we have one action for each of the RESTful actions. The comment before each shows the format of the URL that invokes it.

Notice also that many of the actions contain a `respond_to` block. As we saw in Chapter 11, *Task F: Hotwiring the Storefront*, Rails uses this to determine the type of content to send in a response. The scaffold generator automatically creates code that will respond appropriately to requests for HTML or JSON content. We'll play with that in a little while.

The views created by the generator are fairly straightforward. The only tricky thing is the need to use the correct HTTP method to send requests to the server.

For example, the view for the `index` action looks like this:

rails80/depot_a/app/views/products/index.html.erb

```
<div class="w-full">
  <% if notice.present? %>
    <p class="py-2 px-3 bg-green-50 mb-5 text-green-500 font-medium
              rounded-lg inline-block" id="notice">
      <%= notice %>
    </p>
  <% end %>

  <div class="flex justify-between items-center pb-8">
```